Evaluation of a Distributed E-Commerce Recommendation System

Lorenzo Bartalucci and Lorenzo Marranini
University of Pisa
Cloud Computing Course - A. Y. 2025/2026

1 Introduction

Modern e-commerce platforms generate large amounts of user interaction data, such as product views, cart additions, and purchases. These data can be exploited to build recommendation systems based on users' shopping behaviour.

This project implements a distributed recommendation pipeline using an *item-to-item collaborative filtering* approach. Products appearing together in the same shopping session are considered related, and their co-occurrence frequency is used to generate recommendation lists.

The system is composed of three independent implementations:

- **Java Hadoop MapReduce**: the main distributed solution based on two sequential MapReduce jobs.
- **Apache Spark**: a PySpark implementation reproducing the same recommendation logic.
- **Python Baseline**: a sequential implementation used for correctness verification and performance comparison.

The recommendation workflow follows a simple principle. Each user session is interpreted as a virtual shopping basket containing the products with which the customer interacted. Every pair of distinct products appearing within the same session contributes one co-occurrence observation. After processing all sessions, these observations are aggregated to obtain a co-occurrence score for every product pair. Finally, for each product, only the highest-ranked associated products are retained to build the recommendation list.

2 Dataset

The experiments are based on the **REES46 "eCommerce Behavior Data from a Multi-Category Store"** dataset, publicly available on Kaggle. It contains approximately 42 million anonymized user interactions (5.3 GB) collected during October 2019.

Each record includes the attributes `event_time`, `event_type`, `product_id`, `category_id`, `category_code`, `brand`, `price`, `user_id`, and `user_session`. The recommendation algorithm only uses `user_session`, `product_id`, and `event_type`, retaining only `cart` and `purchase` events.

To evaluate scalability, four dataset slices were generated from the original data, as reported in Table 1.

Table 1: Dataset slices used for the experiments.

Slice	Size
Development	250 MB
Small	500 MB
Medium	1 GB
Large	1.5 GB

All datasets are stored in HDFS under `/user/hadoop/ecommerce/` and are processed by Hadoop MapReduce, Apache Spark, and the Python baseline. During preprocessing, events are grouped by `user_session`, duplicate products are removed, and each session is treated as a virtual shopping basket before generating product co-occurrence pairs.

3 Architecture Description

The system consists of three layers: the **Client Layer**, the **REST Service Layer**, and the **Distributed Processing Layer**. As shown in Figure 1, clients communicate only with the REST service, which submits Hadoop or Spark jobs to the cluster and returns the generated recommendations.

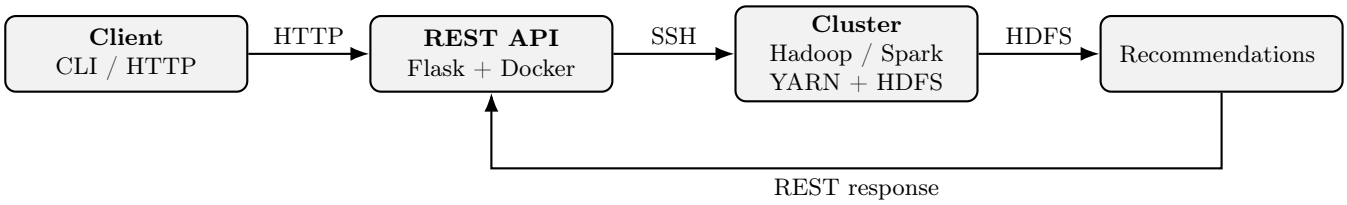


Figure 1: Overall architecture of the distributed recommendation system.

3.1 Distributed Cluster

The distributed computation is executed on a two-node Hadoop cluster. The master node hosts the NameNode, ResourceManager, Spark client and REST service, while the second node acts as a worker running the DataNode and NodeManager services. HDFS provides shared storage for input datasets and generated recommendations.

Table 2 summarizes the cluster configuration.

Table 2: Cluster topology.

Hostname	**IP Address**	**Services**
hadoop-namenode	10.1.1.156	NameNode, ResourceManager, DataNode, NodeManager, Spark Client, REST Server
hadoop-datanode-2	10.1.1.150	DataNode, NodeManager

The software stack includes Hadoop 3.1.3, Apache Spark 3.4.4 running on YARN, Java 8, Ubuntu 22.04 LTS and Docker.

3.2 REST Service Layer

The REST service provides the interface between clients and the Hadoop cluster. Implemented with Flask and deployed in a Docker container, it submits distributed jobs via SSH, monitors their execution, and retrieves the output stored in HDFS.

Requests are handled asynchronously: each submitted job receives a unique identifier that clients can use to check its status and retrieve the final recommendation results.

3.3 Client Layer

The client interacts exclusively with the REST API and does not require Hadoop or Spark to be installed locally. During the experiments, a command-line client was used to submit jobs, monitor their execution, and request recommendations for selected products.

4 Hadoop MapReduce Workflow

The recommendation pipeline is implemented as two consecutive Hadoop MapReduce jobs. Job 1 reconstructs user sessions and generates product co-occurrence pairs, whereas Job 2 aggregates these pairs and computes the Top-N recommendations for each product. The output of the first job is stored temporarily in HDFS and becomes the input of the second stage.

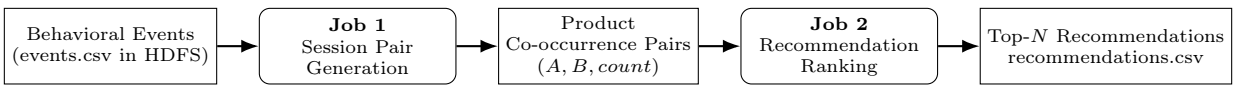


Figure 2: Two-stage Hadoop MapReduce workflow.

Algorithm 1: Job 1 – Session Pair Generation

Input: Behavioral events
Output: Product pairs

Mapper
Read input record;
if *event type* $\notin$ *validEvents* **then**
  discard record;
else
  emit(sessionID, productID);

Reducer
Receive products of the same session;
Remove duplicates using a HashSet;
if *session size* $>$ *maxSessionSize* **then**
  skip session;
else
  generate unordered pairs (p_i, p_j);
  emit(ProductPairWritable(p_i, p_j),1);

Algorithm 2: Job 2 – Recommendation Ranking

Input: Product pairs
Output: Top-N recommendations

Mapper
Read $(A, B, count)$;
emit$(A, (B, count))$;
emit$(B, (A, count))$;

Combiner
Aggregate local counts;

Partitioner
Partition by anchor product;

Reducer
Aggregate global counts;
Maintain a bounded Top-N queue;
Sort by frequency;
emit(anchorProduct,recommendations);

5 Apache Spark Implementation

The recommendation algorithm was also implemented using Apache Spark with the PySpark RDD API. Unlike Hadoop MapReduce, Spark performs most computations in memory, reducing disk I/O and improving execution time for iterative and large-scale data processing.

The implementation follows the same logic described in section 4. Input events are filtered, grouped by `user_session`, and transformed into product co-occurrence pairs. These pairs are then aggregated to compute the Top-N recommendations for each product.

The workflow is implemented through a sequence of RDD transformations, including `filter()`, `map()`, `groupByKey()`, `flatMap()` and `reduceByKey()`, followed by a final sorting stage to retain the highest-ranked recommendations.

Unlike Hadoop, Spark does not require separate Mapper and Reducer classes. The entire pipeline is expressed as a sequence of transformations executed lazily and optimized by the Spark execution engine.

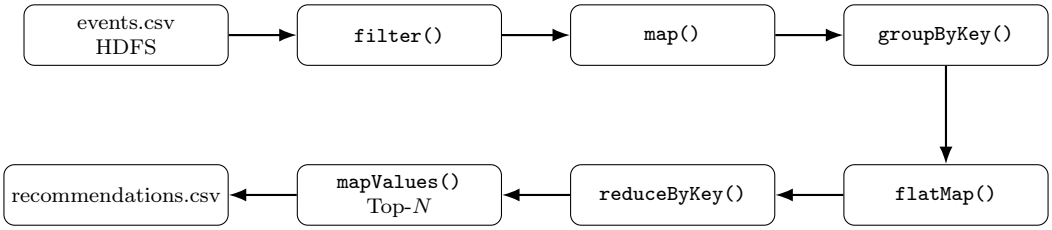


Figure 3: Spark recommendation pipeline implemented through PySpark RDD transformations.

5.1 Comparisons

Table 3: Architectural comparison of the Hadoop MapReduce and Spark implementations.

Architectural Feature	Java Hadoop MapReduce	Apache Spark (RDD)
Execution Model	Two sequential MapReduce jobs executed independently.	Single RDD pipeline executed as a DAG.
Intermediate Data	Job output materialized on HDFS between the two jobs.	Intermediate RDDs retained in memory whenever possible.
Local Aggregation	Explicit Hadoop Combiner performs local pre-aggregation before the Reduce phase.	Aggregation performed through `reduceByKey()`.
Recommendation Ranking	Top-N computed in the Reducer using a bounded priority queue.	Top-N computed through RDD transformations after aggregation.

6 REST Service Architecture

The recommendation system is exposed through a containerized REST service implemented in Python using Flask. The service acts as an intermediate layer between client applications and the distributed Hadoop/Spark cluster. It receives execution requests, submits the corresponding jobs to the cluster via SSH, monitors their execution asynchronously, and retrieves the generated recommendation files from HDFS.

The same REST interface supports the three available execution engines: Hadoop MapReduce, Apache Spark, and the Python baseline. Each submitted job receives a unique identifier that allows clients to monitor its execution and retrieve the generated recommendations once completed.

The REST API exposes the endpoints summarized in Table 4. A lightweight command-line client is also provided to automate job submission, status polling, and recommendation retrieval.

Table 4: REST API endpoints.

Method	Endpoint	Purpose
GET	`/health`	Check service availability.
POST	`/jobs`	Submit a recommendation job.
GET	`/jobs/{id}`	Retrieve job status and execution statistics.
GET	`/jobs/{id}/log`	Retrieve the execution log.
GET	`/jobs/{id}/recommendations/{product}`	Retrieve the Top-N recommendations for a product.

A command-line client interacts with the REST service through standard HTTP requests, providing a simple interface for submitting jobs and retrieving recommendation results independently of the underlying execution engine.

7 Experimental Evaluation & Results

This section evaluates the performance of the Hadoop MapReduce, Apache Spark, and Python baseline implementations. The experiments measure execution time for increasing dataset sizes and analyze the effect of different Hadoop reducers and Spark partitions.

7.1 Experimental Setup

The experiments were executed on the cluster described in Section 3 using dataset subsets ranging from 250 MB to 1.5 GB. Only `cart` and `purchase` events were considered while generating the Top-10 recommendations.

Three experiments were performed:

- **Scaling**: execution time versus dataset size.
- **Reducers**: Hadoop with 1, 2 and 4 reducers.
- **Partitions**: Spark with 2, 4 and 8 partitions.

7.2 Results

Table 5: Execution time for increasing dataset sizes.

Dataset	Hadoop	Spark	Baseline
250 MB	52.7	50.1	4.0
500 MB	59.8	58.9	10.1
1000 MB	67.2	80.0	18.1
1500 MB	76.6	92.2	23.7

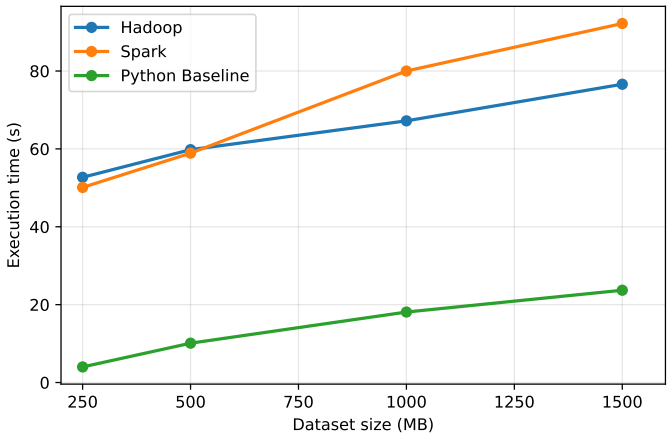
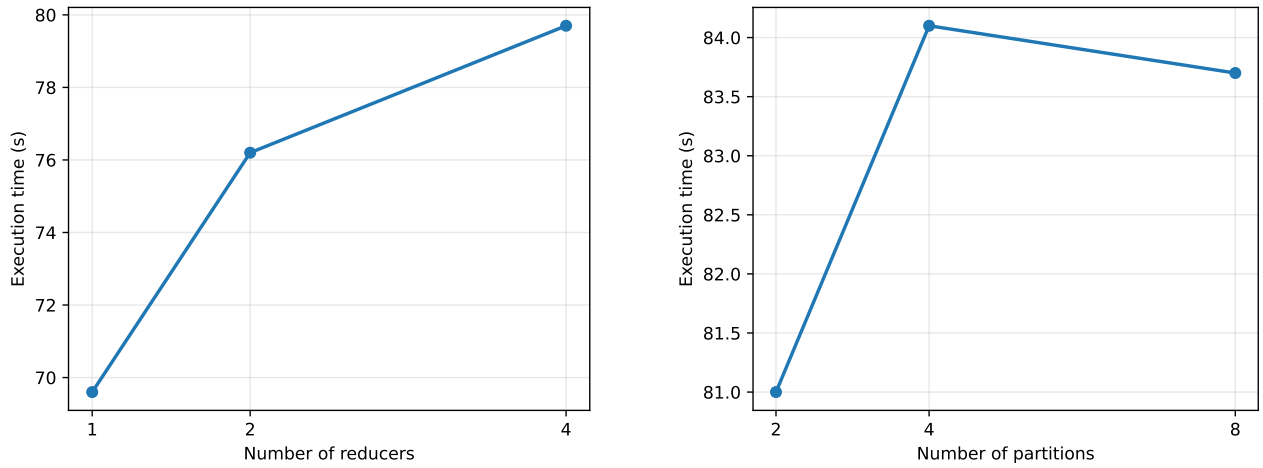


Figure 4: Execution time versus dataset size.

The Python baseline consistently achieves the shortest execution times because it avoids the overhead of distributed execution. Spark is slightly faster than Hadoop on the smaller datasets (250 MB and 500 MB), whereas Hadoop becomes more efficient as the dataset size increases, outperforming Spark on the 1 GB and 1.5 GB datasets.

7.3 Configuration Analysis

The following experiment evaluates the impact of the Hadoop reducer count and the Spark partition configuration on the 1.5 GB dataset.



(a) Hadoop reducers

(b) Spark partitions

Figure 5: Configuration analysis on the 1.5 GB dataset.

For Hadoop, the best execution time is obtained with a single reducer (69.6 s), while increasing the number of reducers to two (76.2 s) and four (79.7 s) introduces additional synchronization overhead. For Spark, using two partitions provides the best execution time (81.0 s), whereas four and eight partitions slightly increase the execution time to 84.1 s and 83.7 s, respectively.

7.4 Conclusions

The experiments highlight the trade-offs between sequential and distributed execution. The Python baseline remains the fastest solution on the evaluated datasets because it avoids cluster coordination and communication overhead.

Among the distributed implementations, Spark performs slightly better on smaller datasets, while Hadoop scales more efficiently for larger inputs under the tested configuration. The configuration analysis also shows that increasing the number of reducers or partitions does not necessarily improve performance, since the additional parallelism may be outweighed by synchronization and scheduling overhead.

Overall, the results demonstrate that selecting an appropriate execution framework and configuration is essential to achieve good performance for the considered workload.